HackFront India 2026 — Team Chaos

Hackathon and Project Overview

Prepared for: Team Chaos
Project: ARCHON — The Engineering Memory Engine
Track: T1 — AI and Developer Tools
Selected problem statement: T1.1 — “Why The Code Is Like This”

---

1. Hackathon Overview

We are participating in **hackFront India 2026** as **Team Chaos**, with Mustafa Pinjari as team lead.

Our chosen theme is **T1: AI and Developer Tools**. The goal is to build a useful developer-focused AI tool that addresses a real engineering problem and can be demonstrated with a working prototype.

Key dates

- **Prototype and cohort enrollment deadline:** October 31, 2026, at 11:59 PM IST
- **Grand Finale:** November 21–22, 2026

Submission checklist

Confirm the latest requirements in the hackathon dashboard before submission. The known submission items are:

- Selected track and problem statement
- Project presentation (PPT or PDF, maximum 25 MB)
- Source-code archive (maximum 50 MB)
- Demo video (MP4 or MOV, maximum 25 MB)
- Public GitHub repository with a useful README
- Optional additional instructions, if needed

The cohort’ s final assessment upload accepts ZIP, RAR, 7Z, TAR, or GZ archives up to 25 MB. Keep this separate from the main hackathon source archive requirement and check the relevant dashboard form before uploading.

2. Selected Problem Statement

T1.1 — “Why The Code Is Like This”

Developers often join an unfamiliar codebase or maintain software whose original design decisions are poorly documented. To understand a piece of code, they may need to inspect source files, Git history, tests, dependencies, and scattered documentation.

The challenge is to build a system that explains **why a codebase may have been designed or changed in a particular way**, attaching a verifiable chain of evidence to every answer.

Important: The project must distinguish evidence-backed facts from interpretations. If the repository does not contain enough evidence, the system should say so instead of inventing an explanation.

---

3. Our Project: ARCHON

ARCHON — The Engineering Memory Engine

ARCHON is an evidence-first engineering intelligence tool that investigates a software repository and helps developers understand code, design decisions, and the possible consequences of changing a component.

One-line pitch:

> Ask a question about an unfamiliar codebase. ARCHON investigates the source code, Git history, tests, and dependencies, then returns an explanation with traceable evidence—or clearly states when evidence is insufficient.

The problem we are solving

When developers maintain unfamiliar software, the reasoning behind implementation choices can be difficult to reconstruct. Relevant clues may be spread across:

- Source code and symbol definitions
- Git commits and code changes
- Tests and test references
- Imports, callers, and module dependencies

- Documentation and other repository artifacts

This creates investigation work and makes it harder to assess the likely impact of a change.

Target users

Our initial intended users are:

- Developers joining an existing project
- Engineers maintaining legacy or poorly documented software
- Technical leads reviewing unfamiliar modules
- Teams trying to understand dependencies before changing code

These are our initial target-user hypotheses. We should validate the problem and workflow with real developer conversations rather than claim that product-market fit is already proven.

---

4. What Makes ARCHON Different?

ARCHON should be more than a generic chatbot that receives a large block of code. Its core differentiator is **evidence-backed repository investigation**.

1. **Repository archaeology:** Search source files and Git history for relevant symbols and changes.
2. **Evidence receipts:** Show the exact source file and line range, commit, test, or dependency that supports a finding whenever available.
3. **Knowledge graph:** Represent relationships among repositories, files, symbols, commits, tests, and evidence.
4. **Evidence-first answers:** Retrieve relevant material first, then generate a bounded explanation from that material.
5. **Change-impact analysis:** Identify potentially affected callers, dependencies, tests, and modules, with suggested verification steps.
6. **Honest uncertainty:** Separate confirmed facts, inferences, hypotheses, and unknowns. Do not fabricate citations, commit history, test coverage, or confidence scores.
7. **Local-first security:** Inspect repositories without executing their code or installing their dependencies. Exclude or redact secrets and sensitive files.

The knowledge graph and AI-generated summaries should only be presented as functional when they are actually implemented and tested. If a feature is a prototype or planned work,

label it accordingly.

---

5. Planned Architecture

The redesign plan is to evolve the working prototype into a product-quality web application while preserving the working version during migration.

Frontend

- **Next.js App Router**
- React and TypeScript
- Tailwind CSS and shadcn/ui
- Lucide icons
- React Flow for interactive relationship graphs
- A focused engineering workspace rather than a generic chat interface

Backend and analysis

- **Python with Django REST Framework**
- Git CLI for repository history and diffs
- Python AST for supported Python symbol and dependency analysis
- Structured, validated API responses
- Pytest for backend tests

Knowledge graph and storage

- **Neo4j** for repository entities and their relationships
- Cypher queries with parameters and bounded traversal
- SQLite for suitable local application metadata
- Idempotent indexing and safe, repository-scoped cleanup

AI and retrieval

- Evidence retrieval before answer generation
- Optional structured LLM summaries behind a provider interface
- A deterministic retrieval path that can still work without a paid AI API
- Optional Neo4j GraphRAG integration if it adds real value and can be verified

Evaluation and quality

- A fixed set of manually verified repository questions
- Tests for citation validity, evidence relevance, unsupported claims, and failure handling
- Honest reporting of the number of evaluated questions and actual results
- Structured logs that avoid recording secrets or unnecessary source content

Architecture status: This is the intended redesign direction, not a claim that every listed component is already implemented.

---

6. Current Prototype Status

The initial working prototype is a **Streamlit application**. The reported functionality includes:

- Local Git repository selection and validation
- Git history searches, including symbol/change searches
- Source-file scanning with extension and file-size limits
- Secret-pattern scrubbing for selected token formats
- Exclusion of sensitive or generated paths such as `.env`, private-key files, and `node_modules`
- Discovery of test references and reverse caller/dependency relationships
- A demo repository clearly labelled as a demo

The project has been tested against a local repository path, including the BAHub project. One investigation returned no explicit matching commits, no test references, and zero evidence receipts for the selected symbol. That result must be treated as **insufficient evidence for that search**, not proof that the symbol has no history or tests. The next step is to select a real symbol with meaningful source references, callers, tests, and/or Git history, then verify every displayed citation against the repository.

These details are based on the current project status reported by the team; confirm them against the latest code and test output before using them as final demo claims.

---

7. Main User Workflow

A successful end-to-end demonstration should follow this flow:

1. **Select a repository** and validate its location.
2. **Ask a focused question**, for example: “Why does this module use this approach, and what could be affected if we change it?”
3. **Identify relevant symbols and files** using source scanning and repository history.
4. **Collect evidence** from source lines, commits, tests, and dependency relationships.
5. **Build or query the relationship graph** for relevant connections.
6. **Generate an evidence-grounded explanation** with citations attached to individual findings.
7. **Show uncertainty and missing evidence** instead of guessing.
8. **Provide change-impact guidance** with potentially affected components and suggested tests.
9. **Let the developer verify the findings** directly against the source and Git history.

For the demo, prefer one complete, reliable investigation over many incomplete features.

---

8. Security and Trust Principles

ARCHON is an analysis tool, not an autonomous code executor.

- Do not run code or scripts from the target repository.
- Do not install target-repository dependencies.
- Validate repository paths and guard against traversal and unsafe symlink handling.
- Apply file-size and file-type limits.
- Exclude or redact secrets and private-key material.
- Avoid sending repository content to external services by default.
- If an external AI provider is enabled, clearly disclose that behavior and make it optional.
- Use parameterized graph queries and repository-scoped cleanup.
- Treat source code, comments, and documentation as untrusted data, not as instructions to the AI.
- Never describe impact analysis as a guarantee that a change is safe.

Security claims should be backed by code review and tests before the team presents them as verified.

---

9. Milestones and Team Collaboration

Suggested build milestones

Milestone 1 — Reliable evidence engine

- Audit the existing working prototype.
- Choose a real symbol in the demo repository.
- Produce verifiable source, history, caller, and test evidence.
- Add tests for missing symbols, absent history, and no-evidence cases.

Milestone 2 — Repository model and graph

- Define stable IDs for files, symbols, commits, tests, and evidence.
- Create the Neo4j schema and constraints.
- Ingest real repository relationships.
- Verify that graph queries return actual repository data.

Milestone 3 — Investigation and answer quality

- Implement the evidence-first investigation pipeline.
- Attach source/commit references to findings.
- Label facts, inferences, hypotheses, and unknowns.
- Test citation validity and unsupported claims.

Milestone 4 — Web application

- Build the Next.js engineering workspace.
- Add Django REST API endpoints only for implemented functionality.
- Keep the existing Streamlit app available until the replacement is working.

Milestone 5 — Evaluation and submission

- Run a small, fixed evaluation set.
- Fix the highest-impact failures.
- Prepare README, architecture diagram, presentation, demo video, source archive, and final walkthrough.
- Recheck all submission limits and deadlines in the official dashboard.

Team working agreement

- Assign an owner and a testable outcome to each task.
- Keep features small enough to verify.
- Review every important AI-generated change.
- Do not claim a feature is complete until it runs and has been tested.
- Record limitations and known failure cases.
- Keep the GitHub repository understandable, with setup steps and a clear demo path.

Suggested division of responsibilities (adapt to the actual team size and skills):

- **Repository intelligence:** Git history, source scanning, symbols, tests, evidence records.
- **Graph and backend:** Neo4j model, ingestion, APIs, validation, tests.
- **Frontend and demo:** investigation workspace, graph view, evidence inspector, presentation.
- **Integration and evaluation:** end-to-end tests, citation checks, documentation, demo script.

One person may own multiple areas. Confirm assignments with the actual team members rather than assuming this division is already agreed.

---

10. Success Criteria

ARCHON is successful for the prototype if a developer can:

- Select a supported local repository safely.
- Ask a specific question about a real code symbol or component.
- Receive findings grounded in actual repository evidence.
- Open or inspect the cited source lines or commit.
- See which statements are facts versus inferences.
- Receive a clear “insufficient evidence” result when the evidence is weak.
- Inspect actual graph relationships if graph functionality is enabled.
- Get a bounded list of potential impacts and verification steps.
- Reproduce the demo using the README instructions.

Potential evaluation measures:

- **Citation validity:** How many displayed references resolve to real source lines or commits?

- **Evidence relevance:** How often does the retrieved evidence actually support the finding?
- **Unsupported-claim rate:** How often does the system make claims not supported by its evidence?
- **Coverage:** How often does it find useful evidence for the selected evaluation questions?
- **Latency and reliability:** How long does an investigation take, and how often does it fail?

Record real results. Do not invent benchmark scores or claim superiority without a fair comparison.

---

11. Risks and Open Questions

- **Weak Git history:** Some repositories are shallow clones, imported archives, or lack meaningful commit messages. The system must degrade gracefully.
- **Language support:** Python AST analysis does not provide complete analysis for every language. Clearly state supported languages and limits.
- **Incomplete call graphs:** Static analysis may miss dynamic calls, reflection, or framework behavior.
- **Graph complexity:** Neo4j is valuable only if the graph improves investigation and can be demonstrated with real queries.
- **LLM hallucinations:** Every answer must be constrained by retrieved evidence and checked for valid citations.
- **Secret exposure:** Repository scanning and any external AI integration need careful boundaries.
- **Time pressure:** Prioritize a complete, credible end-to-end workflow over a large set of half-finished features.
- **Unvalidated demand:** Speak to real developers and record what they currently do, what is painful, and whether ARCHON would fit their workflow.

---

12. What We Should Do Next

1. Confirm the actual hackathon and cohort requirements in the dashboard.
2. Audit the current repository before making large architectural changes.
3. Choose one real investigation question from BAHub or another repository with useful evidence.

4. Make the evidence engine return accurate, clickable source and commit references.
5. Add and run tests for the important success and failure cases.
6. Implement the knowledge graph only with real ingested entities and relationships.
7. Build one polished end-to-end investigation flow in the redesigned UI.
8. Test the demo from a clean setup using the README.
9. Collect feedback from a few developers and update the problem statement based on what is learned.
10. Prepare the final submission materials and verify all size limits and dates.

---

13. Short Pitch for Teammates

ARCHON is an evidence-first engineering memory engine for unfamiliar codebases. It investigates source code, Git history, tests, and dependencies to explain what can be supported about a design or implementation decision. Unlike a generic code chatbot, ARCHON attaches traceable evidence to its findings, distinguishes facts from inferences, and admits when the repository does not provide enough information. Its longer-term design combines repository archaeology, a knowledge graph, evidence-grounded AI, and advisory change-impact analysis.

Our goal is to demonstrate one trustworthy, repeatable workflow—not simply a flashy interface or unsupported AI claims.

Document note: This overview combines the team’s stated hackathon choice, reported prototype status, and intended architecture. Verify current dashboard requirements and implementation status before submission.